Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

Computer Graphics and Animation

BCA — Semester 5 — 2020 — Answer Sheet

Subject Code: CAMG304

Group B

Attempt any SIX questions. [6×5=30]

2. What is computer graphics? Explain different application areas of computer graphics.

[5]

Answer: Computer Graphics: Computer graphics is a field of computer science that deals with generating, manipulating, and displaying visual content using computers. It involves the creation, storage, and manipulation of models and images using mathematical algorithms and computational techniques. **Key Components: • Hardware:** Graphics cards (GPU), displays, input devices • **Software:** Rendering engines, modeling tools, game engines • **Mathematics:** Geometry, linear algebra, calculus for transformations • **Algorithms:** Rasterization, shading, ray tracing, clipping **Application Areas of Computer Graphics:** 1. **Entertainment and Gaming:** - 3D video games (Fortnite, Call of Duty) - CGI in movies (Avatar, Avengers) - Virtual reality experiences - Animation (Pixar, Disney films) 2. **Computer-Aided Design (CAD):** - Architectural design and visualization - Engineering product design (AutoCAD, SolidWorks) - Circuit board layout and design - Manufacturing and prototyping (3D printing) 3. **Medical Imaging:** - MRI, CT scan visualization - 3D reconstruction from 2D slices - Surgical simulation and planning - Virtual anatomy education 4. **Education and Training:** - Interactive educational software - Flight simulators for pilot training - Medical procedure simulators - Virtual laboratories 5. **Scientific Visualization:** - Weather pattern visualization - Molecular modeling in chemistry - Astronomical data representation - Fluid dynamics simulation 6. **Advertising and Marketing:** - Product visualization and mockups - Architectural walkthroughs - Logo and brand design - Virtual showrooms 7. **User Interface Design:** - Graphical user interfaces (GUI) - Icons, buttons, and visual elements - Touchscreen interfaces - AR/VR interfaces 8. **Art and Design:** - Digital painting (Photoshop, Procreate) - Graphic design (Illustrator, CorelDRAW) - Typography and layout design - Generative art using algorithms 9. **Simulation and Virtual Reality:** - Military training simulations - Vehicle crash testing simulation - Urban planning and traffic simulation - Virtual tourism 10. **Data Visualization:** - Business intelligence dashboards - Infographics - Real-time monitoring displays - Geographic information systems (GIS)

3. How can you draw a circle using mid-point circle algorithm? Explain with the algorithm.

[5]

Answer: Mid-Point Circle Algorithm: The mid-point circle algorithm is an efficient rasterization algorithm for drawing circles. It uses the circle's symmetry to calculate only one octant and derives the remaining seven octants by reflection. **Circle Equation:** $x^2 + y^2 = r^2$ **Decision Parameter:** At each step, we evaluate the midpoint between the east (E) and southeast (SE) pixels to decide which pixel to choose next. **Algorithm Steps:** 1. **Initialization:** - Start with x = 0, y = r - Initial decision parameter: P■ = 1 - r (or P■ = 5/4 - r for integer version, approximated as 1 - r) 2. **Plot Initial Point:** - Plot (0, r) and use symmetry to plot all 8 octant points 3. **Iterative Process:** While x < y: a) If P < 0: - Choose East pixel (x+1, y) - P = P + 2x + 3 - Increment x by 1 b) If P ≥ 0: - Choose South-East pixel (x+1, y-1) - P = P + 2(x - y) + 5 - Increment x by 1, decrement y by 1 4. **Symmetry:** For each calculated point (x, y), plot all 8 symmetric points: (x, y), (-x, y), (x, -y), (-x, -y), (y, x), (-y, x), (y, -x), (-y, -x) **Example: Draw circle with radius r = 5** Initial: x = 0, y = 5, P■ = 1 - 5 = -4 StepPAction(x, y)P next 0-4Plot (0,5)(0, 5)- 1-4P<0, E(1, 5)-4+2(0)+3=-1 2-1P<0, E(2, 5)-1+2(1)+3=4 34P≥0, SE(3, 4)4+2(2-5)+5=3 43P≥0, SE(4, 3)3+2(3-4)+5=4 At step 4, x = 4 and y = 3, so x ≥ y. Stop. **Points in first octant:** (0,5), (1,5), (2,5), (3,4), (4,3) **Using symmetry, all circle points are plotted. Advantages:** • Uses only integer arithmetic (fast) • No floating-point calculations needed • Efficient due to symmetry (8-way) • No square root or trigonometric functions **Comparison with DDA:** AspectMid-PointDDA ArithmeticInteger onlyFloating point AccuracyExactAccumulated rounding errors SpeedFasterSlower ComplexityMore complexSimpler

4. Explain 3D basic geometric transformation with an example.

[5]

Answer: 3D Geometric Transformations: 3D transformations are used to modify the position, orientation, or size of 3D objects. They are represented using 4×4 homogeneous transformation matrices for uniform treatment of translation, rotation, and scaling. **1. 3D Translation:** Moves an object by (tx, ty, tz) along each axis. Transformation Matrix: | 1 0 0 tx | | 0 1 0 ty | | 0 0 1 tz | | 0 0 0 1 | A point P(x, y, z, 1) becomes P'(x+tx, y+ty, z+tz, 1) **2. 3D Scaling:** Changes the size of an object by factors (sx, sy, sz). Transformation Matrix: | sx 0 0 0 | | 0 sy 0 0 | | 0 0 sz 0 | | 0 0 0 1 | A point P(x, y, z, 1) becomes P'(x×sx, y×sy, z×sz, 1) If sx = sy = sz = s: Uniform scaling If not equal: Non-uniform scaling **3. 3D Rotation: a) Rotation about X-axis by angle θ:** | 1 0 0 0 | | 0 cosθ -sinθ 0 | | 0 sinθ cosθ 0 | | 0 0 0 1 | P'(x, y×cosθ - z×sinθ, y×sinθ + z×cosθ) **b) Rotation about Y-axis by angle θ:** | cosθ 0 sinθ 0 | | 0 1 0 0 | | -sinθ 0 cosθ 0 | | 0 0 0 1 | P'(x×cosθ + z×sinθ, y, -x×sinθ + z×cosθ) **c) Rotation about Z-axis by angle θ:** | cosθ -sinθ 0 0 | | sinθ cosθ 0 0 | | 0 0 1 0 | | 0 0 0 1 | P'(x×cosθ - y×sinθ, x×sinθ + y×cosθ, z) **Example: Translate a point P(2, 3, 4) by (5, -2, 3) and then scale by (2, 2, 2) Translation:** P' = T × P P' = (2+5, 3-2, 4+3) = (7, 1, 7) **Scaling:** P'' = S × P' P'' = (7×2, 1×2, 7×2) = (14, 2, 14) **Combined Transformation:** For multiple transformations, matrices are multiplied in reverse order of application. If we want to Scale then Translate: P' = T × S × P T × S = | 1 0 0 5 | | 2 0 0 0 | | 2 0 0 5 | | 0 1 0 -2 | × | 0 2 0 0 | = | 0 2 0 -2 | | 0 0 1 3 | | 0 0 2 0 | | 0 0 2 3 | | 0 0 0 1 | | 0 0 0 1 | | 0 0 0 1 | P' = (2×2+5, 3×2-2, 4×2+3) = (9, 4, 11) **Properties:** • Matrix multiplication is associative but not commutative • Order of transformations matters (T then S ≠ S then T) • Homogeneous coordinates (4th coordinate = 1) allow translation to be expressed as matrix multiplication

5. What is polygon clipping? Explain Sutherland Hodgman algorithm for polygon clipping.

[5]

Answer: Polygon Clipping: Polygon clipping is the process of removing parts of a polygon that lie outside a specified clipping region (usually a rectangular window). It is essential in computer graphics for displaying only the visible portion of objects and for operations like hidden surface removal. **Why Line Clipping is Insufficient:** Line clipping algorithms (Cohen-Sutherland, Liang-Barsky) only handle individual line segments. When clipping polygons, we need to maintain the polygon structure and generate new edges where the polygon crosses the clipping boundary. **Sutherland-Hodgman Polygon Clipping Algorithm:** This algorithm clips a polygon against a convex clipping window by processing each edge of the window sequentially. It clips the polygon against one edge at a time, producing a new vertex list after each edge. **Algorithm:** 1. **Input:** List of polygon vertices (in order) and clipping window edges 2. **Process each window edge:** For each edge of the clipping window (left, right, top, bottom): a) Initialize output vertex list as empty b) For each polygon edge (from current vertex S to next vertex P): Case 1: **S inside, P inside** - Add P to output list Case 2: **S inside, P outside** - Add intersection point I to output list Case 3: **S outside, P outside** - Add nothing Case 4: **S outside, P inside** - Add intersection point I and P to output list c) Use output list as input for next window edge 3. **Output:** Final clipped polygon vertices **Four Cases Illustrated:** Inside side Outside side S ■■■■■■■■● I (Case 2: S in, P out → add I) S ■■■■■■■■→ P (Case 1: S in, P in → add P) I ●←■■■■■■■ P (Case 4: S out, P in → add I, P) S ■■■■■■■■→ P (Case 3: S out, P out → add nothing) (both outside) **Example: Clip polygon against left edge (x = x_min)** For each edge SP of polygon: • S_inside = S.x ≥ x_min • P_inside = P.x ≥ x_min • Intersection I: y = S.y + (P.y - S.y) × (x_min - S.x) / (P.x - S.x) **Advantages:** • Simple and easy to implement • Works for any convex clipping window • Processes one edge at a time **Limitations:** • Only works for convex clipping regions • For concave polygons, may produce extra edges • Does not handle self-intersecting polygons **Comparison with Weiler-Atherton:** AspectSutherland-HodgmanWeiler-Atherton Window typeConvex onlyAny (convex/concave) Polygon typeAnyAny ComplexitySimplerMore complex OutputSingle polygonMultiple polygons possible **Applications:** Viewport clipping in 2D graphics, shadow volume clipping, CSG operations

6. Given a triangle with vertices A(2,3), B(5,5), C(4,3) by rotating 90 degree about the origin and then translating two unit in each direction. Use homogenous transformation matrix to find the new vertices of the triangle.

[5]

Answer: Given: • Triangle vertices: A(2,3), B(5,5), C(4,3) • Rotation: 90° about origin (counter-clockwise) • Translation: 2 units in x and 2 units in y **Step 1: Rotation Matrix (90° about origin, counter-clockwise)** R(90°) = | cos90° -sin90° 0 | | 0 -1 0 | | sin90° cos90° 0 | = | 1 0 0 | | 0 0 1 | | 0 0 1 | For counter-clockwise rotation by θ: x' = x×cosθ - y×sinθ y' = x×sinθ + y×cosθ At θ = 90°: cos90° = 0, sin90° = 1 **Apply Rotation to each vertex:** A(2, 3): A'x = 2×0 - 3×1 = -3 A'y = 2×1 + 3×0 = 2 **A' = (-3, 2)** B(5, 5): B'x = 5×0 - 5×1 = -5 B'y = 5×1 + 5×0 = 5 **B' = (-5, 5)** C(4, 3): C'x = 4×0 - 3×1 = -3 C'y = 4×1 + 3×0 = 4 **C' = (-3, 4) Step 2: Translation Matrix (tx = 2, ty = 2)** T = | 1 0 2 | | 0 1 2 | | 0 0 1 | **Step 3: Combined Transformation** First rotate, then translate: M = T × R M = | 1 0 2 | | 0 -1 0 | | 0 -1 2 | | 0 1 2 | × | 1 0 0 | = | 1 0 2 | | 0 0 1 | | 0 0 1 | | 0 0 1 | **Step 4: Apply Combined Transformation** For A(2, 3): | 0 -1 2 | | 2 | | 0×2 + (-1)×3 + 2×1 | | -1 | | 1 0 2 | × | 3 | = | 1×2 + 0×3 + 2×1 | = | 4 | | 0 0 1 | | 1 | | 0×2 + 0×3 + 1×1 | | 1 | **A'' = (-1, 4)** For B(5, 5): | 0 -1 2 | | 5 | | 0×5 + (-1)×5 + 2 | | -3 | | 1 0 2 | × | 5 | = | 1×5 + 0×5 + 2 | = | 7 | | 0 0 1 | | 1 | | 0×5 + 0×5 + 1 | | 1 | **B'' = (-3, 7)** For C(4, 3): | 0 -1 2 | | 4 | | 0×4 + (-1)×3 + 2 | | -1 | | 1 0 2 | × | 3 | = | 1×4 + 0×3 + 2 | = | 6 | | 0 0 1 | | 1 | | 0×4 + 0×3 + 1 | | 1 | **C'' = (-1, 6) Final Answer:** VertexOriginalAfter RotationAfter Translation A(2, 3)(-3, 2)(-1, 4) B(5, 5)(-5, 5)(-3, 7) C(4, 3)(-3, 4)(-1, 6) **New vertices of the triangle: A''(-1, 4), B''(-3, 7), C''(-1, 6)**

7. Explain the scan line algorithm for visible surface detection.

[5]

Answer: Scan Line Algorithm for Visible Surface Detection: The scan line algorithm is an image-space method for hidden surface removal. It processes the scene one horizontal scan line at a time, determining which surfaces are visible at each pixel along the scan line. **Basic Principle:** For each horizontal scan line across the viewport: 1. Determine which polygons intersect the scan line 2. Find intersection points of these polygons with the scan line 3. Sort intersection points by x-coordinate 4. Determine which segments are visible based on depth 5. Draw visible segments with appropriate colors **Data Structures Used:** 1. **Edge Table (ET):** - Contains all non-horizontal edges of all polygons - Grouped by the y-coordinate of the edge's lower endpoint - Each entry stores: y_max, x_intersect (at lower y), inverse slope (1/m) 2. **Active Edge Table (AET):** - Contains edges that intersect the current scan line - Updated as the algorithm moves from one scan line to the next - Sorted by x-intersection coordinate **Algorithm Steps:** 1. **Initialize:** - Build Edge Table for all polygons - Set y = minimum y-coordinate in the scene - Initialize AET as empty 2. **For each scan line from bottom to top:** a) Move edges from ET[y] to AET (edges starting at this y) b) Remove edges from AET where y = y_max (edges ending at this y) c) Sort AET by x-intersection coordinate d) Process pairs of x-intersections in AET: - Between each pair of x-values, determine which polygon is visible - Compare z-depths of polygons at sample points - Draw visible polygon's color for that segment e) Update x-intersections for next scan line: - x_new = x_old + 1/m (increment by inverse slope) f) Increment y by 1 3. **Repeat until all scan lines are processed. Example with Two Polygons:** Polygon 1 (Blue): closer to viewer Polygon 2 (Red): farther from viewer Scan line y = 50: ■■■■ ET adds edges that start at y=50 ■■■■ AET contains: Edge1 (P1), Edge2 (P1), Edge3 (P2), Edge4 (P2) ■■■■ Sorted AET by x: x1, x2, x3, x4 ■■■■ Segments: [x1,x2]=P1 visible, [x2,x3]=neither, [x3,x4]=P2 visible ■■■■ Draw blue from x1 to x2, red from x3 to x4 **Handling Overlapping Polygons:** When multiple polygons cover the same pixel: 1. Calculate z-depth of each polygon at that pixel 2. The polygon with smallest z-value (closest to viewer) is visible 3. If depths are equal, use additional criteria (arbitrary choice or painter's algorithm) **Advantages:** • Works at image resolution (pixel level) • Can handle complex overlapping polygons • Efficient for scenes with many polygons • Can be combined with shading (Gouraud, Phong) **Disadvantages:** • Computationally expensive per pixel • Requires maintaining and sorting edge tables • Aliasing problems at edges (jagged lines) • Difficult to handle transparent surfaces **Optimizations:** • Span coherence: Adjacent pixels often have same visibility • Edge coherence: x-intersections change predictably between scan lines • Depth coherence: Depth changes linearly across a polygon face **Comparison with Z-Buffer:** AspectScan LineZ-Buffer SpaceLess memory (edge tables)More memory (depth buffer) SpeedFaster for simple scenesConsistent for complex scenes ImplementationMore complexSimpler Anti-aliasingEasierHarder

8. Explain the architecture of VR system with necessary components.

[5]

Answer: Virtual Reality (VR) System Architecture: A VR system creates an immersive, computer-generated environment that simulates physical presence in a real or imagined world. The architecture consists of hardware components, software systems, and user interfaces working together. **1. Input Devices:** a) **Head-Mounted Display (HMD) Sensors:** - Accelerometers, gyroscopes, magnetometers (IMU) - Track head position and orientation (6 DOF) - Examples: Oculus Rift, HTC Vive, PlayStation VR b) **Hand Controllers:** - Track hand position and gestures - Provide haptic feedback (vibrations) - Examples: Oculus Touch, Valve Index controllers c) **Motion Tracking Systems:** - Outside-in tracking: External base stations/lighthouses - Inside-out tracking: Cameras on HMD itself - Marker-based tracking: Reflective markers on user/body d) **Additional Input:** - Eye tracking (foveated rendering) - Voice recognition - Body suits and treadmills - Bio-sensors (heart rate, brain activity) **2. Output Devices:** a) **Head-Mounted Display (HMD):** - Dual displays (one per eye) for stereoscopic 3D - High refresh rate (90-120 Hz minimum to prevent motion sickness) - Wide field of view (100-110° typical) - Lens systems for focus and distortion correction b) **Audio Systems:** - 3D spatial audio (head-related transfer function) - Directional sound matching visual cues - Noise cancellation for immersion c) **Haptic Feedback:** - Vibrations in controllers and suits - Force feedback devices - Treadmills for walking sensation **3. Computing System:** a) **Rendering Engine:** - Real-time 3D graphics rendering (OpenGL, Vulkan, DirectX) - Foveated rendering (high quality where user looks) - Asynchronous timewarp/spacewarp for smooth frame delivery b) **Tracking and Processing:** - Sensor fusion algorithms (Kalman filters) - Predictive tracking to reduce latency - World reconstruction and spatial mapping c) **Application Logic:** - Game engines: Unity, Unreal Engine - Physics simulation - AI for virtual characters and environments **4. Software Architecture Layers:** +----------------------------------+ | VR Applications & Content | +----------------------------------+ | VR SDK / Game Engine | | (Unity XR, OpenXR, SteamVR) | +----------------------------------+ | VR Runtime / Compositor | | (Oculus Runtime, SteamVR) | +----------------------------------+ | Device Drivers & APIs | | (OpenVR, OpenXR, OSVR) | +----------------------------------+ | Operating System | | (Windows, Linux, Android) | +----------------------------------+ | Hardware (GPU, CPU, USB) | +----------------------------------+ **5. Key Technical Challenges:** a) **Latency:** - Motion-to-photon latency must be < 20ms to prevent motion sickness - Achieved through prediction, timewarp, high refresh rates b) **Resolution and Field of View:** - Higher resolution reduces screen-door effect - Wider FOV increases immersion - Trade-off with GPU performance c) **Comfort:** - Ergonomic HMD design - Adjustable IPD (interpupillary distance) - Weight distribution - Ventilation to prevent fogging **6. Types of VR Systems:** • **Non-immersive (Desktop VR):** 3D environment on standard screen • **Semi-immersive:** Large projection systems, flight simulators • **Fully Immersive:** HMD with tracking, most common today • **Collaborative VR:** Multiple users in shared virtual space • **Augmented Reality (AR):** Overlays virtual objects on real world • **Mixed Reality (MR):** Interactive blend of real and virtual **Applications:** Gaming, education, medical training, architecture, military simulation, virtual tourism, therapy and rehabilitation

Group C

Attempt any TWO questions. [2×10=20]

9. Explain the working details of DDA line drawing algorithm? Digitize the line with endpoints (2,2) and (10,5) using Bresenham's line drawing algorithm.

[5+5]

Answer: DDA (Digital Differential Analyzer) Line Drawing Algorithm: DDA is a scan-conversion algorithm based on calculating either dx or dy using the line equation. It samples the line at unit intervals in one coordinate and calculates the corresponding integer value nearest to the line path for the other coordinate. **Algorithm:** 1. Calculate dx = x2 - x1 and dy = y2 - y1 2. Determine steps = max(|dx|, |dy|) 3. Calculate x_increment = dx / steps 4. Calculate y_increment = dy / steps 5. Set x = x1, y = y1 6. Plot pixel at (round(x), round(y)) 7. For i = 1 to steps: - x = x + x_increment - y = y + y_increment - Plot pixel at (round(x), round(y)) **Example: Line from (2, 2) to (10, 5)** dx = 10 - 2 = 8 dy = 5 - 2 = 3 steps = max(8, 3) = 8 x_increment = 8/8 = 1.0 y_increment = 3/8 = 0.375 ixyPlot (round) 02.02.0(2, 2) 13.02.375(3, 2) 24.02.75(4, 3) 35.03.125(5, 3) 46.03.5(6, 4) 57.03.875(7, 4) 68.04.25(8, 4) 79.04.625(9, 5) 810.05.0(10, 5) **Advantages:** Simple to implement **Disadvantages:** Floating-point operations, rounding errors, slower than Bresenham --- **Bresenham's Line Drawing Algorithm:** Bresenham's algorithm uses only integer arithmetic to determine the next pixel position. It evaluates a decision parameter at each step to choose between two candidate pixels. **Given:** Endpoints (2, 2) and (10, 5) **Step 1: Calculate differences** dx = 10 - 2 = 8 dy = 5 - 2 = 3 **Step 2: Check slope** Since |dx| > |dy| and dx, dy > 0: 0 < slope < 1, we increment x by 1 at each step. **Step 3: Initialize decision parameter** P■ = 2×dy - dx = 2×3 - 8 = 6 - 8 = -2 **Step 4: Decision rules** • If P < 0: Next point = (x+1, y), P = P + 2×dy • If P ≥ 0: Next point = (x+1, y+1), P = P + 2×dy - 2×dx **Step 5: Iteration** Step(x, y)PDecisionNext P 0(2, 2)-2P<0, E-2+6=4 1(3, 2)4P≥0, NE4+6-16=-6 2(4, 3)-6P<0, E-6+6=0 3(5, 3)0P≥0, NE0+6-16=-10 4(6, 4)-10P<0, E-10+6=-4 5(7, 4)-4P<0, E-4+6=2 6(8, 4)2P≥0, NE2+6-16=-8 7(9, 5)-8P<0, E-8+6=-2 8(10, 5)-2End- **Pixels plotted by Bresenham:** (2, 2), (3, 2), (4, 3), (5, 3), (6, 4), (7, 4), (8, 4), (9, 5), (10, 5) **Note:** Both DDA and Bresenham produce the same set of pixels for this line, but Bresenham uses only integer arithmetic, making it faster.

10. Write the difference between object space method and image space method. Explain Z-buffer algorithm for visible surface detection.

[5+5]

Answer: Object Space vs Image Space Methods: AspectObject Space MethodImage Space Method Processing LevelWorks with 3D object coordinatesWorks with 2D pixel coordinates ComparisonCompares objects with each otherCompares pixels depth values PrecisionExact (geometric calculations)Approximate (pixel resolution) ComplexityO(n^2) where n = number of objectsO(n×p) where p = number of pixels OutputDetermines visible surfaces directlyDetermines visible pixels Anti-aliasingDifficultEasier MemoryLess memory requiredMore memory (depth buffer) ExamplesBack-face removal, BSP trees, ray castingZ-buffer, scan line, ray tracing Scene SizeBetter for fewer objectsBetter for complex scenes AccuracyIndependent of resolutionLimited by image resolution **Object Space Methods:** • **Back-Face Removal:** Eliminate polygons facing away from viewer • **Ray Casting:** Cast rays from eye through pixels, find first intersection • **Depth Sorting (Painter's Algorithm):** Sort objects by depth, paint back to front **Image Space Methods:** • **Z-Buffer (Depth Buffer):** Store depth at each pixel • **Scan Line:** Process one scan line at a time • **A-Buffer:** Anti-aliased version of Z-buffer --- **Z-Buffer Algorithm (Depth Buffer Algorithm): Principle:** The Z-buffer algorithm maintains two buffers: 1. **Frame Buffer:** Stores color/intensity of each pixel 2. **Z-Buffer (Depth Buffer):** Stores z-coordinate (depth) of visible surface at each pixel **Algorithm:** 1. **Initialize:** - Frame buffer: Set to background color - Z-buffer: Set to maximum depth value (farthest from viewer) 2. **For each polygon in the scene:** a) For each pixel (x, y) covered by the polygon: i. Calculate depth z of polygon at (x, y) - For a plane ax + by + cz + d = 0: - z = -(ax + by + d) / c ii. If z < Z-buffer[x, y]: - Z-buffer[x, y] = z - Frame buffer[x, y] = polygon color at (x, y) 3. **Output:** Frame buffer contains the final image **Depth Calculation Optimization:** Instead of calculating z for each pixel from scratch: • For a scan line: z(x+1, y) = z(x, y) - a/c • For next scan line: z(x, y+1) = z(x, y) - b/c This incremental calculation is very efficient. **Example:** Two polygons at a pixel: - Polygon A: z = 5 (farther) - Polygon B: z = 2 (closer) Processing A first: Z-buffer = 5, Frame buffer = Color_A Processing B: z = 2 < Z-buffer (5) Z-buffer = 2, Frame buffer = Color_B Final pixel shows Color_B (closer polygon) **Advantages:** • Simple to implement in hardware • Handles any polygon complexity • Works for overlapping/intersecting polygons • Linear time complexity O(number of pixels × polygons) • Can be parallelized efficiently **Disadvantages:** • Requires large memory for Z-buffer (e.g., 1920×1080 × 4 bytes ≈ 8 MB) • Limited precision (z-fighting when depths are very close) • Cannot handle transparency (requires multiple passes or A-buffer) • Wastes processing on hidden surfaces • Aliasing at polygon edges **Variants:** • **W-Buffer:** Uses 1/z for better precision distribution • **Hierarchical Z-Buffer:** Uses coarse Z values to reject blocks of pixels early • **Reverse Z:** Maps near plane to 1.0 and far plane to 0.0 for better precision

11. Derive the formula for windows to viewport transformation. Given a window bordered by (0,0) at the lower-left and (4,4) at the upper right. Similarly, a viewport bordered by (0,0) at the lower-left and (2,2) at the upper right. If a window at position (2,4) is mapped into the viewport. What will be the position of viewport to maintain same relative placement as in window?

[5+5]

Answer: Window to Viewport Transformation: The window defines what portion of the world coordinate system is visible. The viewport defines where on the display device the window contents appear. The transformation maps coordinates from the window to the viewport while maintaining relative positions. **Derivation:** Let: • Window: (x_wmin, y_wmin) to (x_wmax, y_wmax) • Viewport: (x_vmin, y_vmin) to (x_vmax, y_vmax) • Point in window: (x_w, y_w) • Corresponding point in viewport: (x_v, y_v) **For X-coordinate:** We want the relative position of x_w within the window to equal the relative position of x_v within the viewport: (x_w - x_wmin) / (x_wmax - x_wmin) = (x_v - x_vmin) / (x_vmax - x_vmin) Solving for x_v: **x_v = x_vmin + (x_w - x_wmin) × (x_vmax - x_vmin) / (x_wmax - x_wmin) For Y-coordinate:** Similarly: (y_w - y_wmin) / (y_wmax - y_wmin) = (y_v - y_vmin) / (y_vmax - y_vmin) Solving for y_v: **y_v = y_vmin + (y_w - y_wmin) × (y_vmax - y_vmin) / (y_wmax - y_wmin) General Formula:** x_v = x_vmin + (x_w - x_wmin) × Sx y_v = y_vmin + (y_w - y_wmin) × Sy Where: • Sx = (x_vmax - x_vmin) / (x_wmax - x_wmin) [Scaling factor for x] • Sy = (y_vmax - y_vmin) / (y_wmax - y_wmin) [Scaling factor for y] **Matrix Form (including normalization):** | x_v | | Sx 0 tx | | x_w | | y_v | = | 0 Sy ty | × | y_w | | 1 | | 0 0 1 | | 1 | Where: • tx = x_vmin - x_wmin × Sx • ty = y_vmin - y_wmin × Sy --- **Given Problem: Window:** • Lower-left: (x_wmin, y_wmin) = (0, 0) • Upper-right: (x_wmax, y_wmax) = (4, 4) **Viewport:** • Lower-left: (x_vmin, y_vmin) = (0, 0) • Upper-right: (x_vmax, y_vmax) = (2, 2) **Point in window:** (x_w, y_w) = (2, 4) **Step 1: Calculate Scaling Factors** Sx = (x_vmax - x_vmin) / (x_wmax - x_wmin) = (2 - 0) / (4 - 0) = 2/4 = **0.5** Sy = (y_vmax - y_vmin) / (y_wmax - y_wmin) = (2 - 0) / (4 - 0) = 2/4 = **0.5 Step 2: Calculate Translation Terms** tx = x_vmin - x_wmin × Sx = 0 - 0 × 0.5 = **0** ty = y_vmin - y_wmin × Sy = 0 - 0 × 0.5 = **0 Step 3: Apply Transformation** x_v = x_vmin + (x_w - x_wmin) × Sx x_v = 0 + (2 - 0) × 0.5 = 2 × 0.5 = **1** y_v = y_vmin + (y_w - y_wmin) × Sy y_v = 0 + (4 - 0) × 0.5 = 4 × 0.5 = **2 Verification of Relative Placement:** • In window: (2, 4) is at 50% of x-range (2/4) and 100% of y-range (4/4) • In viewport: (1, 2) is at 50% of x-range (1/2) and 100% of y-range (2/2) The relative positions are maintained. **Final Answer:** The point (2, 4) in the window maps to **(1, 2)** in the viewport. **Visualization:** Window (4×4) Viewport (2×2) y y ↑ ↑ 4 ■■■■■ (2,4)● 2 ■■■■■■■●(1,2) 3 ■ 1 ■ 2 ■ 0 ■■■■■ 1 ■ 0 ■■■■■→ x → x 0 1 2 3 4 0 1 2

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.